

Chapter 5: Top-Down Parsing

陳奇業 成功大學資訊工程系





Objectives of Top-Down Parsing

- an attempt to find a leftmost derivation for an input string.
- an attempt to construct a parse tree for the input string starting from the root and creating the nodes of the parse tree in preorder.



Objectives of Top-Down Parsing

- In this chapter, we study the following two forms of top-down parsers:
 - **Recursive-descent parsers** contain a set of mutually recursive procedures that cooperate to parse a string. Code for these procedures can be written directly from a suitable grammar.
 - **Table-driven LL parsers** use a generic LL(k) parsing engine and a parse table that directs the activity of the engine. The entries for the parse table are determined by the particular LL(k) grammar. The notation LL(k) is explained below.

The Basic Method of Recursive-Descent

$exp \rightarrow exp \text{ addop } term \mid term$

$addop \rightarrow + \mid -$

$term \rightarrow term \text{ mulop } factor \mid factor$

$mulop \rightarrow *$

$factor \rightarrow (exp) \mid \text{number}$

```
procedure factor ;  
begin  
  case token of  
    ( : match( ( ) ;  
      exp ;  
      match( ) ) ;  
    number :  
      match(number) ;  
    else error ;  
  end case ;  
end factor ;
```

Using EBNF

- Consider now the case of an *exp* in the grammar for simple arithmetic expressions in BNF:
 $exp \rightarrow exp \text{ addop } term \mid term$
- The solution is to use the EBNF rule
 $exp \rightarrow term \{ addop term \}$

```
procedure exp ;  
begin  
    term ;  
    while token = + or token = - do  
        match (token) ;  
        term ;  
    end while ;  
end exp ;
```

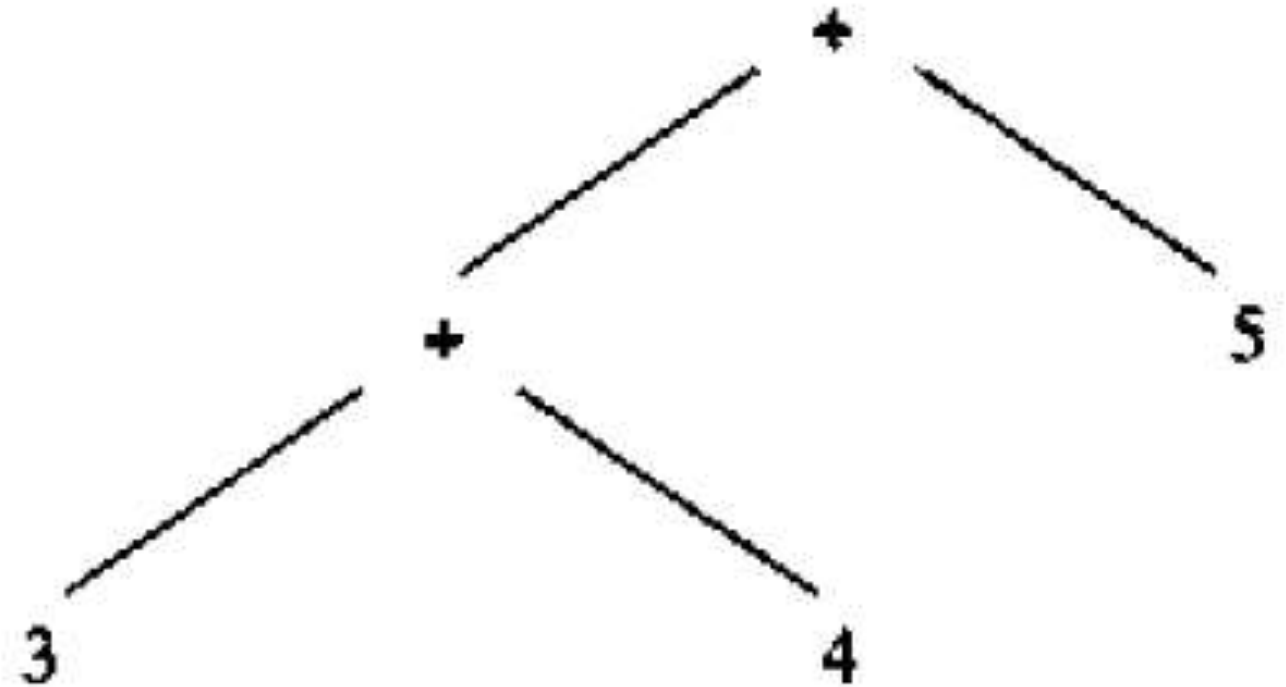
Syntax Tree

```
function exp : integer ;  
var temp : integer ;  
begin  
    temp := term ;  
    while token = + or token = - do  
        case token of  
            + : match (+) ;  
                temp := temp + term ;  
            - : match (-) ;  
                temp := temp - term ;  
        end case ;  
    end while ;  
    return temp ;  
end exp ;
```

```
function exp : syntaxTree ;  
var temp, newtemp : syntaxTree ;  
begin  
    temp := term ;  
    while token = + or token = - do  
        newtemp := makeOpNode(token) ;  
        match (token) ;  
        leftChild(newtemp) := temp ;  
        rightChild(newtemp) := term ;  
        temp := newtemp ;  
    end while ;  
    return temp ;  
end exp ;
```

Syntax Tree

- We consider the expression $3+4+5$





Approaches of Top-Down Parsing

with backtracking (making repeated scans of the input, a general form of top-down parsing)

Methods: To create a procedure for each nonterminal.

Problems for top-down parsing with backtracking

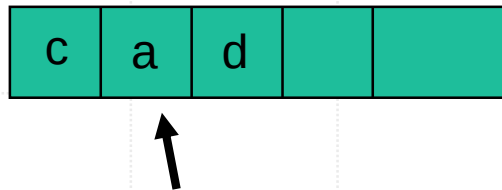
```
void A() {  
1)    Choose an  $A$ -production,  $A \rightarrow X_1 X_2 \cdots X_k$ ;  
2)    for (  $i = 1$  to  $k$  ) {  
3)        if (  $X_i$  is a nonterminal )  
4)            call procedure  $X_i()$ ;  
5)        else if (  $X_i$  equals the current input symbol  $a$  )  
6)            advance the input to the next symbol;  
7)        else /* an error has occurred */;  
    }  
}
```

e.g. $S \rightarrow cAd$ $A \rightarrow ab \mid a$

$L = \{ cabd, cad \}$

```
S( ) { if input symbol == 'c'
      { Advance();
        if A()
          if input-symbol == 'd'
            { Advance();
              return true;
            }
        }
      }
    return false;
}
```

```
A( ) { isave = input-pointer;
      if input-symbol == 'a'
        { Advance();
          if input-symbol == 'b'
            { Advance();
              return true;
            }
        }
      input-pointer = isave;
      if input-symbol == 'a'
        { Advance();
          return true; }
      else
        return false;
    }
```





Problems for top-down parsing with backtracking

- left-recursion (can cause a top-down parser to go into an infinite loop)
 - Def. A grammar is said to be left-recursive if it has a nonterminal A s.t. there is a derivation $A \Rightarrow A\delta$ for some δ .
- backtracking - undo not only the movement but also the semantics entering in symbol table.
- the order the alternatives are tried (For the grammar shown above, try $w = cabd$ where $A \rightarrow a$ is applied first)

The LL(1) Predict Function

- Given the productions

$A \rightarrow \alpha_1$

$A \rightarrow \alpha_2$

...

$A \rightarrow \alpha_n$

- During a (leftmost) derivation

$\dots A \dots \Rightarrow \dots \alpha_1 \dots$ **or**

$\dots \alpha_2 \dots$ **or**

$\dots$ **or**

$\dots \alpha_n \dots$

- Deciding which production to match
 - Using lookahead symbols

The LL(1) Predict Function

Single Symbol Lookahead

$$\text{Predict}(A \rightarrow X_1 \cdots X_m) = \begin{cases} (\text{First}(X_1 \cdots X_m) - \lambda) \cup \text{Follow}(A), & \text{if } \lambda \in \text{First}(X_1 \cdots X_m) \\ \text{First}(X_1 \cdots X_m) & , \text{otherwise} \end{cases}$$

- The limitation of LL(1)
 - LL(1) contains exactly those grammars that have disjoint predict sets for productions that share a common left-hand side



A grammar G is LL(1) if and only if whenever $A \rightarrow \alpha | \beta$ are two distinct productions of G , the following conditions hold:

- 1. For no terminal a do both α and β derive strings beginning with a .
- 2. At most one of α and β can derive the empty string.
- 3. If $\beta \Rightarrow^* \epsilon$ then α does not derive any string beginning with a terminal in $\text{FOLLOW}(A)$.
Likewise, if $\alpha \Rightarrow^* \epsilon$, then β does not derive any string beginning with a terminal in $\text{FOLLOW}(A)$

The LL(1) Predict Function

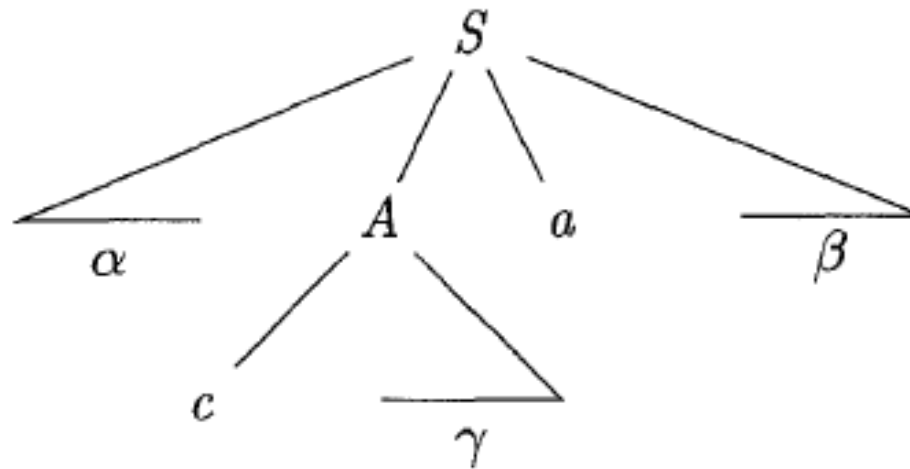


Figure 4.15: Terminal c is in $\text{FIRST}(A)$ and a is in $\text{FOLLOW}(A)$



First set

- To compute $\text{First}(X)$ for all grammar symbols X , apply the following rules until no more terminals or λ can be added to any First set.
 1. If X is a terminal, then $\text{First}(X) = \{X\}$.
 2. If X is a nonterminal and $X \rightarrow Y_1 Y_2 \cdots Y_k$ is a production for some $k \geq 1$, then place a in $\text{First}(X)$ if for some i , a is in $\text{First}(Y_i)$, and λ is in all of $\text{First}(Y_1), \dots, \text{First}(Y_{i-1})$; that is $Y_1 \cdots Y_{i-1} \Rightarrow^* \lambda$. If λ is in $\text{First}(Y_j)$ for all $j = 1, 2, \dots, k$, then add λ to $\text{First}(X)$.
 3. If $X \rightarrow \lambda$ is a production, then add λ to $\text{First}(X)$.

An Example

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \lambda$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \lambda$$

$$F \rightarrow (E) \mid id$$

$$First(E) = First(T) = First(F) = \{ (, id \}$$

$$First(E') = \{ +, \lambda \}$$

$$First(T') = \{ *, \lambda \}$$

Consider our simple integer expression grammar:²

$$\begin{aligned} \text{exp} &\rightarrow \text{exp addop term} \mid \text{term} \\ \text{addop} &\rightarrow + \mid - \\ \text{term} &\rightarrow \text{term mulop factor} \mid \text{factor} \\ \text{mulop} &\rightarrow * \\ \text{factor} &\rightarrow (\text{exp}) \mid \text{number} \end{aligned}$$

We write out each choice separately so that we may consider them in order (we also number them for reference):

- (1) $\text{exp} \rightarrow \text{exp addop term}$
- (2) $\text{exp} \rightarrow \text{term}$
- (3) $\text{addop} \rightarrow +$
- (4) $\text{addop} \rightarrow -$
- (5) $\text{term} \rightarrow \text{term mulop factor}$
- (6) $\text{term} \rightarrow \text{factor}$
- (7) $\text{mulop} \rightarrow *$
- (8) $\text{factor} \rightarrow (\text{exp})$
- (9) $\text{factor} \rightarrow \text{number}$

$\text{First}(\text{exp}) = \{ (, \text{number})$

$\text{First}(\text{term}) = \{ (, \text{number})$

$\text{First}(\text{factor}) = \{ (, \text{number})$

$\text{First}(\text{addop}) = \{ +, - \}$

$\text{First}(\text{mulop}) = \{ * \}$



Follow set

- To compute $\text{Follow}(A)$ for all nonterminals A , apply the following rules until nothing can be added to any Follow set.
 1. Place λ in $\text{Follow}(S)$, where S is the start symbol.
 2. If there is a production $A \rightarrow \alpha B \beta$, then everything in $\text{First}(\beta)$ except λ is in $\text{Follow}(B)$.
 3. If there is a production $A \rightarrow \alpha B$, or a production $A \rightarrow \alpha B \beta$, where $\text{First}(\beta)$ contains λ , then everything in $\text{Follow}(A)$ is in $\text{Follow}(B)$.

An Example

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \lambda$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \lambda$$

$$F \rightarrow (E) \mid id$$

/ E is the start symbol */*

$$\text{Follow}(E) = \{\lambda,)\} \quad // \text{ rules 1 \& 2}$$

$$\text{Follow}(E') = \{\lambda,)\} \quad // \text{ rule 3}$$

$$\text{Follow}(T) = \{+, \lambda,)\} \quad // \text{ rules 2 \& 3}$$

$$\text{Follow}(T') = \{+, \lambda,)\} \quad // \text{ rule 3}$$

$$\text{Follow}(F) = \{*, +, \lambda,)\} \quad // \text{ rules 2 \& 3}$$

$$\text{First}(E) = \text{First}(T) = \text{First}(F) = \{(, id\}$$

$$\text{First}(E') = \{+, \lambda\}$$

$$\text{First}(T') = \{*, \lambda\}$$

The LL(1) Predict Function

- A grammar G is LL(1) if and only if whenever $A \rightarrow \alpha | \beta$ are two distinct productions of G , the following conditions hold:

1. For no terminal a do both α and β derive strings beginning with a .
 $\text{First}(\alpha) \cap \text{First}(\beta) = \phi$
2. At most one of α and β can derive the empty string.
3. If $\beta \Rightarrow^* \lambda$, then α does not derive any string beginning with a terminal in $\text{Follow}(A)$.
Likewise, if $\alpha \Rightarrow^* \lambda$, then β does not derive any string beginning with a terminal in $\text{Follow}(A)$.

common prefixes



$\text{First}(\alpha) \cap \text{Follow}(A) = \phi$ (i.e. If $\text{First}(\alpha)$ contains λ then $\text{First}(\beta) \cap \text{Follow}(A) = \phi$)

1	<program>	→ begin <statement list> end	
2	<statement list>	→ <statement> <statement tail>	
3	<statement tail>	→ <statement> <statement tail>	
4	<statement tail>	→ λ	
5	<statement>	→ ID := <expression> ;	
6	<statement>	→ read (<id list>) ;	
7	<statement>	→ write (<expr list>) ;	
8	<id list>	→ ID <id tail>	
9	<id tail>	→ , ID <id tail>	
10	<id tail>	→ λ	
11	<expr list>	→ <expression> <expr tail>	
12	<expr tail>	→ , <expression> <expr tail>	
13	<expr tail>	→ λ	
14	<expression>	→ <primary> <primary tail>	
15	<primary tail>	→ <add op> <primary> <primary tail>	
16	<primary tail>	→ λ	
17	<primary>	→ (<expression>)	
18	<primary>	→ ID	
19	<primary>	→ INTLIT	
20	<add op>	→ +	<i>Not extended BNF form</i>
21	<add op>	→ -	
22	<system goal>	→ <program> \$	\$: end of file token

Figure 5.1 A Micro Grammar in Standard Form

The LL(1) Parse Table

- An LL(1) parse table
 $T: V_n \times V_t \rightarrow P \cup \{\text{Error}\}$
- The definition of T

$T[A][t] = A \rightarrow X_1 \cdots X_m$ if t
 $\in \text{Prediction}(A \rightarrow X_1 \cdots X_m);$

$T[A][t] = \text{Error}$, otherwise

	ID	INTLIT	:=	,	;	+	-	(	)	begin	end	read	write	\$
<program>										1				
<statement list>	2											2	2	
<statement>	5											6	7	
<statement tail>	3										4	3	3	
<expression>	14	14						14						
<id list>	8													
<expr list>	11	11						11						
<id tail>				9					10					
<expr tail>				12					13					
<primary>	18	19						17						
<primary tail>				16	16	15	15		16					
<add op>						20	21							
<system goal>										22				

Figure 5.5 The LL(1) Table for Micro

1	<program>	→ begin <statement list> end
2	<statement list>	→ <statement> <statement tail>
3	<statement tail>	→ <statement> <statement tail>
4	<statement tail>	→ λ
5	<statement>	→ ID := <expression> ;
6	<statement>	→ read (<id list>) ;
7	<statement>	→ write (<expr list>) ;
8	<id list>	→ ID <id tail>
9	<id tail>	→ , ID <id tail>
10	<id tail>	→ λ
11	<expr list>	→ <expression> <expr tail>
12	<expr tail>	→ , <expression> <expr tail>
13	<expr tail>	→ λ
14	<expression>	→ <primary> <primary tail>
15	<primary tail>	→ <add op> <primary> <primary tail>
16	<primary tail>	→ λ
17	<primary>	→ (<expression>)
18	<primary>	→ ID
19	<primary>	→ INTLIT
20	<add op>	→ +
21	<add op>	→ -
22	<system goal>	→ <program> \$

Figure 5.1 A Micro Grammar in Standard Form

Nonterminal	First Set
<program>	{ begin }
<statement list>	{ID, read , write }
<statement>	{ID, read , write }
<statement tail>	{ID, read , write , λ}
<expression>	{ID, INTLIT, (}
<id list>	{ID}
<expr list>	{ID, INTLIT, (}
<id tail>	{COMMA, λ}
<expr tail>	{COMMA, λ}
<primary>	{ID, INTLIT, (}
<primary tail>	{+, −, λ}
<add op>	{+, −}
<system goal>	{ begin }

Figure 5.2 First Sets for Micro

NonTerminal	Follow Set
<program>	{ \$ }
<statement list>	{ end }
<statement>	{ID, read , write , end }
<statement tail>	{ end }
<expression>	{COMMA, SEMICOLON,)}
<id list>	{ } }
<expr list>	{ } }
<id tail>	{ } }
<expr tail>	{ } }
<primary>	{COMMA, SEMICOLON, +, −,)}
<primary tail>	{COMMA, SEMICOLON,)}
<add op>	{ID, INTLIT, (}
<system goal>	{λ}

Figure 5.3 Follow Sets for Micro

1 <program> → **begin** <statement list> **end**
 2 <statement list> → <statement> <statement tail>
 3 <statement tail> → <statement> <statement tail>
 4 <statement tail> → λ
 5 <statement> → ID := <expression> ;
 6 <statement> → **read** (<id list>) ;
 7 <statement> → **write** (<expr list>) ;
 8 <id list> → ID <id tail>
 9 <id tail> → , ID <id tail>
 10 <id tail> → λ
 11 <expr list> → <expression> <expr tail>
 12 <expr tail> → , <expression> <expr tail>
 13 <expr tail> → λ
 14 <expression> → <primary> <primary tail>
 15 <primary tail> → <add op> <primary> <primary tail>
 16 <primary tail> → λ
 17 <primary> → (<expression>)
 18 <primary> → ID

Figure 5

Nonterminal	First Set
<program>	{ begin }
<statement list>	{ID, read , write }
<statement>	{ID, read , write }
<statement tail>	{ID, read , write , λ }
<expression>	{ID, INTLIT, {}}
<id list>	{ID}
<expr list>	{ID, INTLIT, {}}
<id tail>	{COMMA, λ }
<expr tail>	{COMMA, λ }
<primary>	{ID, INTLIT, {}}
<primary tail>	{+, -, λ }
<add op>	{+, -}
<system goal>	{ begin }

Figure 5.2 First Sets for Micro

Prod	Predict Set		
1	First(begin <statement list> end) =	First(begin) =	{ begin }
2	First(<statement> <statement tail>) =	First(<statement>) =	{ID, read , write }
3	First(<statement> <statement tail>) =	First(<statement>) =	{ID, read , write }
4	$(\text{First}(\lambda) - \lambda) \cup \text{Follow}(\text{<statement tail>}) =$	$\text{Follow}(\text{<statement tail>}) =$	{ end }
5	First(ID := <expression> ;) =	First(ID) =	{ID}
6	First(read (<id list>) ;) =	First(read) =	{ read }
7	First(write (<expr list>) ;) =	First(write) =	{ write }
8	First(ID <id tail>) =	First(ID) =	{ID}
9	First(, ID <id tail>) =	First(,) =	{,}
10	$(\text{First}(\lambda) - \lambda) \cup \text{Follow}(\text{<id tail>}) =$	$\text{Follow}(\text{<id tail>}) =$	{}
11	First(<expression> <expr tail>) =	First(<expression>) =	{ID, INTLIT, {}}
12	First(, <expression> <expr tail>) =	First(,) =	{,}
13	$(\text{First}(\lambda) - \lambda) \cup \text{Follow}(\text{<expr tail>}) =$	$\text{Follow}(\text{<expr tail>}) =$	{}
14	First(<primary> <primary tail>) =	First(<primary>) =	{ID, INTLIT, {}}
15	First(<add op> <primary> <primary tail>) =	First(<add op>) =	{+, -}
16	$(\text{First}(\lambda) - \lambda) \cup \text{Follow}(\text{<primary tail>}) =$	$\text{Follow}(\text{<primary tail>}) =$	{COMMA, ,, }
17	First((<expression>)) =	First(() =	{(}
18	First(ID) =		{ID}
19	First(INTLIT) =		{INTLIT}
20	First(+) =		{+}
21	First(-) =		{-}
22	First(<program> \$) =	First(<program>) =	{ begin }

Figure 5.4 Calculation of Predict Sets for Micro

Building Recursive Descent Parsers from LL(1) Tables

- The form of parsing procedure:

```
void non_term(void)
{
    token tok = next_token();
    switch (tok) {
    case TERMINAL_LIST:
        parsing_actions();
        break;
        . . .
    default:
        syntax_error(tok);
        break;
    }
}
```

Building Recursive Descent Parsers from LL(1) Tables

- E.g. of an parsing procedure for **<statement>** in Micro
- An algorithm that automatically creates parsing procedures like the one in Figure 5.6 from LL(1) table

```
void statement(void)
{
    token tok;
    tok = next_token();
    switch (tok) {
    case ID:
        match(ID); match(ASSIGNOP); expression();
        match(SEMICOLON);
        break;

    case READ:
        match(READ); match(LPAREN); id_list();
        match(RPAREN); match(SEMICOLON);
        break;

    case WRITE:
        match(WRITE); match(LPAREN); expr_list();
        match(RPAREN); match(SEMICOLON);
        break;

    default:
        syntax_error(tok);
        break;
    }
}
```

Figure 5.6 Parsing Procedure for <statement>

Building Recursive Descent Parsers from LL(1) Tables

- The data structure for describing grammars

```
typedef int symbol;    /* a symbol in the grammar */

#define VOCABULARY    (NUM_NONTERMINALS + NUM_TERMINALS)

typedef struct gram {
    symbol terminals[NUM_TERMINALS];
    symbol nonterminals[NUM_NONTERMINALS];
    symbol start_symbol;
    int num_productions;
    struct prod {
        symbol lhs;
        int rhs_length;
        symbol rhs[MAX_RHS_LENGTH];
    } productions[NUM_PRODUCTIONS];
    symbol vocabulary[VOCABULARY];
    char *names[VOCABULARY];
} grammar;

typedef struct prod production;

typedef symbol terminal;
typedef symbol nonterminal;
```

Building Recursive Descent Parsers from LL(1) Tables

- `gen_actions()`
 - Takes the grammar symbols and generates the actions necessary to match them in a recursive descent parse

```
extern char *make_id(char *);

void gen_actions(symbol x[], int x_length)
{
    int i;
    char *id;

    /*
     * Generate recursive descent
     * actions needed to match x.
     */
    if (x_length == 0)
        printf("; /* null */\n");
    else {
        for (i = 0; i < x_length; i++) {
            id = make_id(g.names[x[i]]);
            if (is_terminal(x[i]))
                printf("\t\tmatch(%s);\n", id);
            else
                printf("\t\t%s();\n", id);
        }
    }
}
```

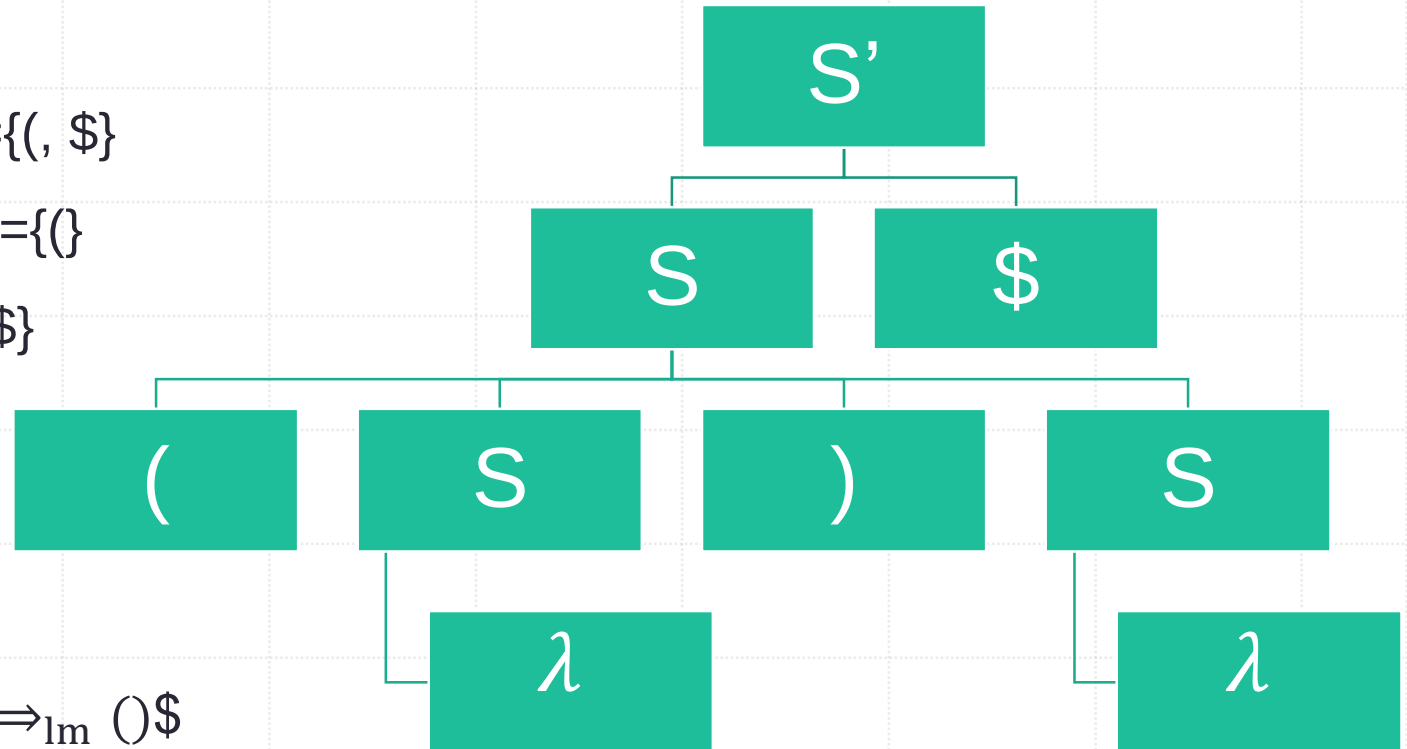
Figure 5.7 Algorithm to Generate Recursive Descent Actions

```
void make_parsing_proc(const nonterminal A,  
                      const lltable T)  
{  
    /*  
     * Generate recursive descent  
     * parsing procedure for A.  
     */  
    extern grammar g;  
    production p;  
    terminal x;  
    int i, j;  
  
    printf("void %s(void)\n{\n", make_id(g.names[A]));  
    printf("\ttoken tok = next_token()\n");  
    printf("\tswitch (tok) {\n");  
  
    /* for each production where A is the LHS */  
    for (i = 0; i < g.num_productions; i++) {  
        if (g.productions[i].lhs != A)  
            continue;  
        p = g.productions[i];  
        /* for each terminal in the grammar */  
        for (j = 0; j < NUM_TERMINALS; j++) {  
            x = g.terminals[j];  
            if (T[A][x] == i) /* this production */  
                printf("\tcase %s:\n", make_id(g.names[x]));  
        }  
        gen_actions(p.rhs, p.rhs_length);  
        printf("\t\tbreak;\n");  
    }  
    printf("\tdefault:\n");  
    printf("\t\tsyntax_error(tok);\n");  
    printf("\t\tbreak;\n}\n\n");  
}
```

Figure 5.8 Algorithm to Generate Parsing Procedures

LL(1) Parsing

- $S' \rightarrow S\$$ $\text{PredictSet}(S' \rightarrow S\$) = \{ (, \$ \}$
- $S \rightarrow (S)S$ $\text{PredictSet}(S \rightarrow (S)S) = \{ (\}$
- $S \rightarrow \lambda$ $\text{PredictSet}(S \rightarrow \lambda) = \{ \$ \}$
- Input String: $()\$$



-
- $S' \Rightarrow_{lm} S\$ \Rightarrow_{lm} (S)S\$ \Rightarrow_{lm} ()S\$ \Rightarrow_{lm} ()\$$



Elimination of Left Recursion

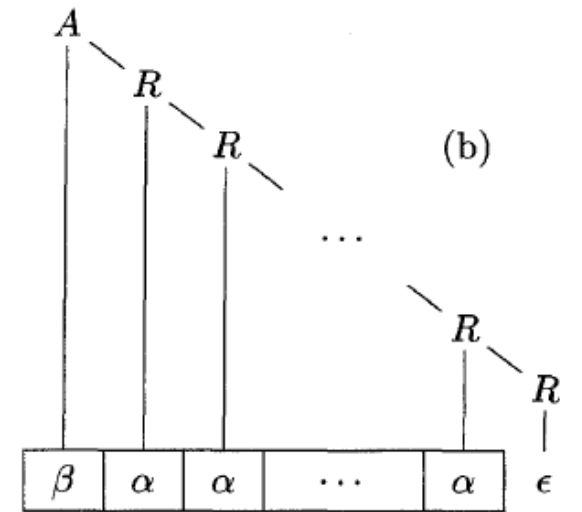
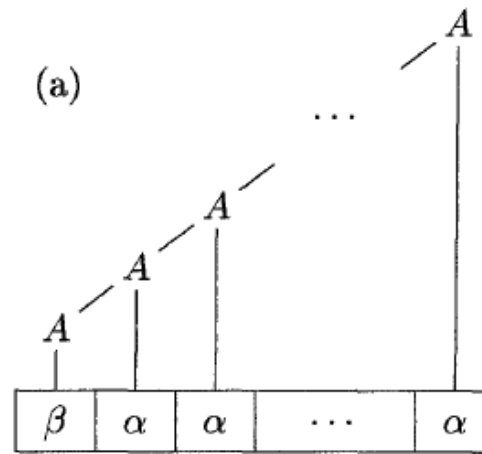
- It is possible for a recursive-descent parser to loop forever. A problem arises with "left-recursive" productions like
 $\text{expr} \rightarrow \text{expr} + \text{term}$
- A left-recursive production can be eliminated by rewriting the offending production.
Consider a nonterminal A with two productions
 $A \rightarrow A\alpha | \beta$
- For example, $A = \text{expr}$, $\alpha = + \text{term}$, $\beta = \text{term}$

Elimination of Left Recursion

- We can convert left recursion to right recursion in the following manner, using a new nonterminal R :

$$A \rightarrow \beta R$$

$$R \rightarrow \alpha R \mid \epsilon$$



Elimination of Immediate Left Recursion

- Immediate left recursion can be eliminated by the following technique, which works for any number of A -productions. First, group the productions as

$$A \rightarrow A\alpha_1 | A\alpha_2 | \cdots | A\alpha_m | \beta_1 | \beta_2 | \cdots | \beta_n$$

where no β_i begins with an A . Then, replace the A -productions by

$$\begin{aligned} A &\rightarrow \beta_1 A' | \beta_2 A' | \cdots | \beta_n A' \\ A' &\rightarrow \alpha_1 A' | \alpha_2 A' | \cdots | \alpha_m A' | \lambda \end{aligned}$$

e.g.

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

After transformation:

$$E \rightarrow TE' \quad E' \rightarrow +TE' \mid \lambda$$

$$T \rightarrow FT' \quad T' \rightarrow *FT' \mid \lambda$$

$$F \rightarrow (E) \mid id$$



How about left recursion occurred for derivation with more than two steps?

e.g., $S \rightarrow Aa \mid b \quad A \rightarrow Ac \mid Sd \mid e$

where $S \Rightarrow Aa \Rightarrow Sda$

Algorithm: Eliminating left recursion

Input: Context-free Grammar G with no cycles or λ -production

Methods:

1. Arrange the nonterminals in some order $A_1, A_2, \dots, A_n$
2. for $i = 1$ to n do
 - {
 - for $j = 1$ to $i - 1$ do
 - {
 - replace each production of the form $A_i \rightarrow A_j \gamma$ by the production $A_i \rightarrow \delta_1 \gamma | \delta_2 \gamma | \dots | \delta_k \gamma$, where $A_j \rightarrow \delta_1 | \delta_2 | \dots | \delta_k$ are all current A_j -production;
 - }
 - eliminate the immediate left-recursion among the A_i -production;
 - }

#We say a grammar G is cyclic if for some nonterminal A in G , we have $A \Rightarrow^+ A$



An Example

e.g.

$$S \rightarrow Aa \mid b$$

$$A \rightarrow Ac \mid Sd \mid e$$

$$\text{Step 1: } \Rightarrow S \rightarrow Aa \mid b$$

$$\text{Step 2: } \Rightarrow A \rightarrow Ac \mid Aad \mid bd \mid e$$

$$\text{Step 3: } \Rightarrow A \rightarrow bdA' \mid eA' \quad A' \rightarrow cA' \mid adA' \mid \varepsilon$$

Non-backtracking (recursive-descent) parsing

recursive descent : use a collection of mutually recursive routines to perform the syntax analysis.

Left Factoring : $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \implies A \rightarrow \alpha A' \quad A' \rightarrow \beta_1 \mid \beta_2$

Methods:

1. For each nonterminal A find the **longest prefix** α common to two or more of its alternatives.
If $\alpha \neq \lambda$ replace all the A productions

$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \text{others}$ by $A \rightarrow \alpha A' \mid \text{others} \quad A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$

2. Repeat the transformation until no more found

e.g. $S \rightarrow iCtS \mid iCtSeS \mid a \quad C \rightarrow b$

$\implies S \rightarrow iCtSS' \mid a \quad S' \rightarrow eS \mid \lambda \quad C \rightarrow b$



Predicative Parsing

Features:

- maintains a stack rather than recursive calls
- table-driven

Components:

1. An input buffer with end marker (\$)
2. A stack with start symbol (S') on the bottom
~~A stack with endmarker (\$) on the bottom~~ (龍書)
3. A parsing table, a two-dimensional array $M[A, a]$, where ' A ' is a nonterminal symbol and ' a ' is the current input symbol (terminal/token).

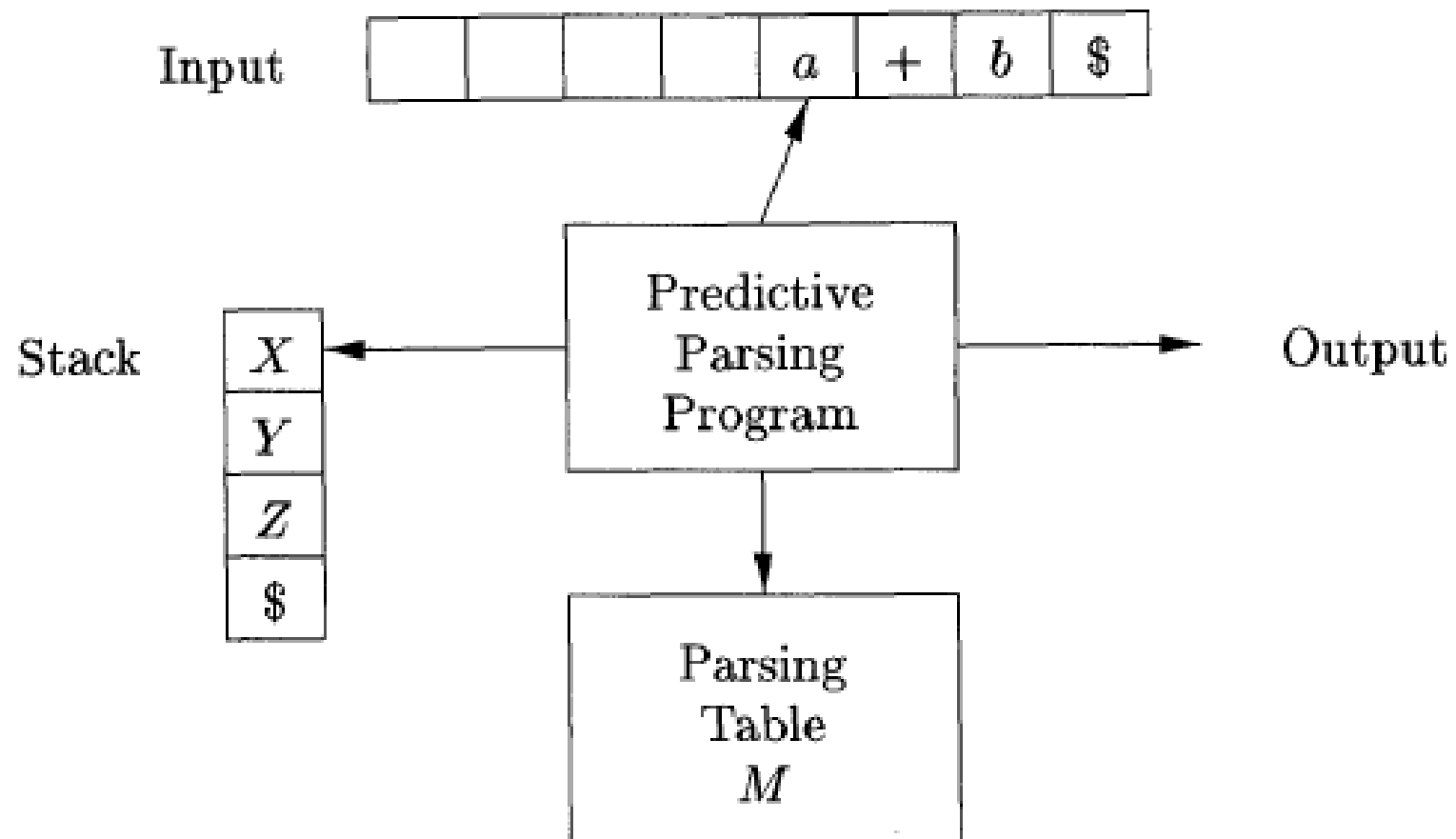


Figure 4.19: Model of a table-driven predictive parser

Parsing Table

$S' \rightarrow S\$$
 $S \rightarrow (S)S$
 $S \rightarrow \lambda$

$M[A, a]$	$(\quad) \quad \$$		
S	$S \rightarrow (S)S$	$S \rightarrow \lambda$	$S \rightarrow \lambda$
S'	$S' \rightarrow S\$$		$S' \rightarrow S\$$



Algorithm:

Input: An input string w and a parsing table M for grammar G .

Output: A leftmost derivation of w or an error indication.

Initially $w\$$ is in input buffer and S' (龍書是 $S\$$) is in the stack.

 Starting Symbol of the grammar

Method:

```
do { Let  $a$  of  $w$  be the next input symbol and  $X$  be the top stack symbol;
    if  $X$  is a terminal
        { if  $X == a$  then pop  $X$  from stack and remove  $a$  from input;
          else ERROR();}
    else
        { if  $M[X, a] = X \rightarrow Y_1 Y_2 \cdots Y_n$  then
            1. pop  $X$  from the stack;
            2. push  $Y_n Y_{n-1} \cdots Y_1$  onto the stack with  $Y_1$  on top;
          else
            ERROR();
        }
    } while ( $X \neq \$$ )
if ( $X == \$$ ) and (the next input symbol ==  $\$$ ) then accept else error();
```

Figure 4.2

Table-based LL(1) parsing
algorithm

```
(* assumes $ marks the bottom of the stack and the end of the input *)
push the start symbol onto the top of the parsing stack ;
while the top of the parsing stack  $\neq$  $ and the next input token  $\neq$  $ do
    if the top of the parsing stack is terminal  $a$ 
        and the next input token =  $a$ 
    then (* match *)
        pop the parsing stack ;
        advance the input ;
    else if the top of the parsing is nonterminal  $A$ 
        and the next input token is terminal  $a$ 
        and parsing table entry  $M[A, a]$  contains
            production  $A \rightarrow X_1 X_2 \dots X_n$ 
    then (* generate *)
        pop the parsing stack ;
        for  $i := n$  downto 1 do
            push  $X_i$  onto the parsing stack ;
        else error ;
if the top of the parsing stack = $
    and the next input token = $
then accept
else error ;
```

Table 4.1

Parsing actions of a
top-down parser

	Parsing stack	Input	Action
1	$\$ S$	$() \$$	$S \rightarrow (S) S$
2	$\$ S) S ($	$() \$$	match
3	$\$ S) S$	$) \$$	$S \rightarrow \epsilon$
4	$\$ S)$	$) \$$	match
5	$\$ S$	$\$$	$S \rightarrow \epsilon$
6	$\$$	$\$$	accept

$S' \rightarrow S\$$
 $S \rightarrow (S)S$
 $S \rightarrow \lambda$

$M[A, a]$	$($	$)$	$\$$
S	$S \rightarrow (S) S$	$S \rightarrow \lambda$	$S \rightarrow \lambda$
S'	$S' \rightarrow S\$$		$S' \rightarrow S\$$

Table 4.2

LL(1) parsing table for
(ambiguous) if-statements

$M[N, T]$	if	other	else	0	1	\$
<i>statement</i>	<i>statement</i> $\rightarrow$ <i>if-stmt</i>	<i>statement</i> $\rightarrow$ other				
<i>if-stmt</i>	<i>if-stmt</i> $\rightarrow$ if (<i>exp</i>) <i>statement</i> <i>else-part</i>					
<i>else-part</i>			<i>else-part</i> $\rightarrow$ else <i>statement</i> <i>else-part</i> $\rightarrow \epsilon$			<i>else-part</i> $\rightarrow \epsilon$
<i>exp</i>				<i>exp</i> $\rightarrow$ 0	<i>exp</i> $\rightarrow$ 1	

Start $\rightarrow$ **statement** **\$**

First(state) = {if, other}

First(if-stmt) = {if}

First(else-part) = {else, ϵ }

First(exp) = {0, 1}

Follow(state) = {\$, else}

Follow(if-stmt) = {\$, else}

Follow(else-part) = {\$, else}

Follow(exp) = {}



Construct a Predicative Parsing Table

1. For each production $A \rightarrow \alpha$ of the grammar, do steps 2 and 3.
2. For each terminal a in $\text{First}(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$.
3. If λ is in $\text{First}(\alpha)$, add $A \rightarrow \alpha$ to $M[A, b]$ for each terminal b in $\text{Follow}(A)$.
4. Make each undefined entry of M be error.



LL(1) grammar

A grammar whose parsing table has no multiply-defined entries is said to be LL(1).

First 'L' : scan the input from left to right.

Second 'L' : produce a leftmost derivation.

'1' : use one input symbol to determine parsing action.

* No ambiguous or left-recursive grammar can be LL(1).

Def. for Multiply-defined entry

If G is left-recursive or ambiguous, then M will have at least one multiply-defined entry.

e.g.

$S \rightarrow iCtSS' \mid a \quad S' \rightarrow eS \mid \lambda \quad C \rightarrow b$

generates:

$M[S', e] = \{ S \rightarrow \lambda, S' \rightarrow eS \}$ with multiply- defined entry.

Start $\rightarrow S \$$

$\text{First}(S) = \{i, a\}, \text{First}(S') = \{e, \lambda\}, \text{First}(C) = \{b\}$

$\text{Follow}(S) = \{\$, e\}, \text{Follow}(S') = \{\$, e\}, \text{Follow}(C) = \{t\}$

Parsing table with multiply-defined entry

	a	b	e	i	t	$\$$
S	$S \rightarrow a$			$S \rightarrow iCtSS'$		
S'			$S' \rightarrow \lambda$ $S' \rightarrow eS$			$S' \rightarrow \lambda$
C		$C \rightarrow b$				

$S \rightarrow iCtSS' \mid a \quad S' \rightarrow eS \mid \lambda \quad C \rightarrow b$

Start $\rightarrow S \$$

$\text{First}(S)=\{i, a\}, \text{First}(S')=\{e, \lambda\}, \text{First}(C)=\{b\}$

$\text{Follow}(S)=\{\$, e\}, \text{Follow}(S')=\{\$, e\}, \text{Follow}(C)=\{t\}$